

Arial 54pt

Arial 44pt

Arial 24pt

Arial 14pt

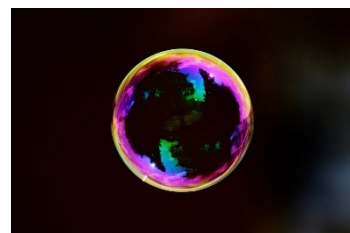
Arial 10pt

Arial 9pt

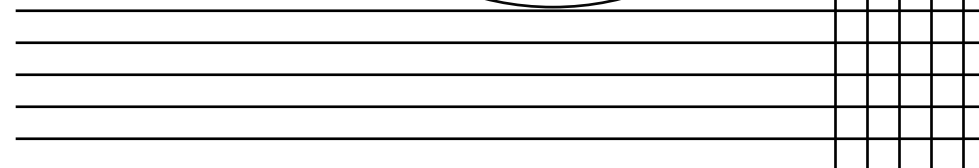
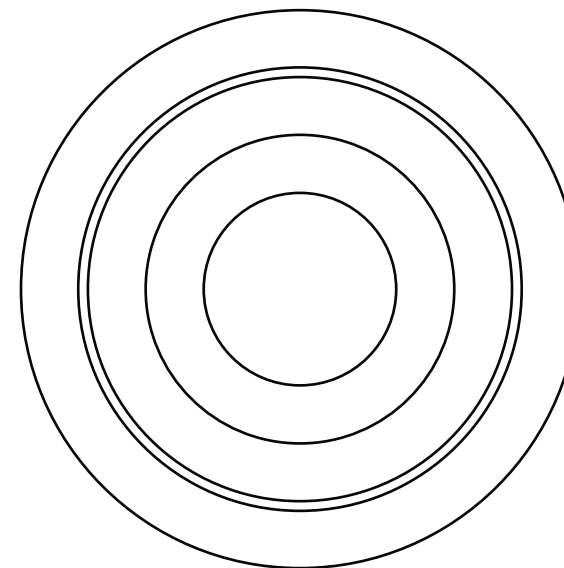
SimpleStd-Bold

Whitman

Trebuchet



JR Signature Template 0.1



Lecture Slides

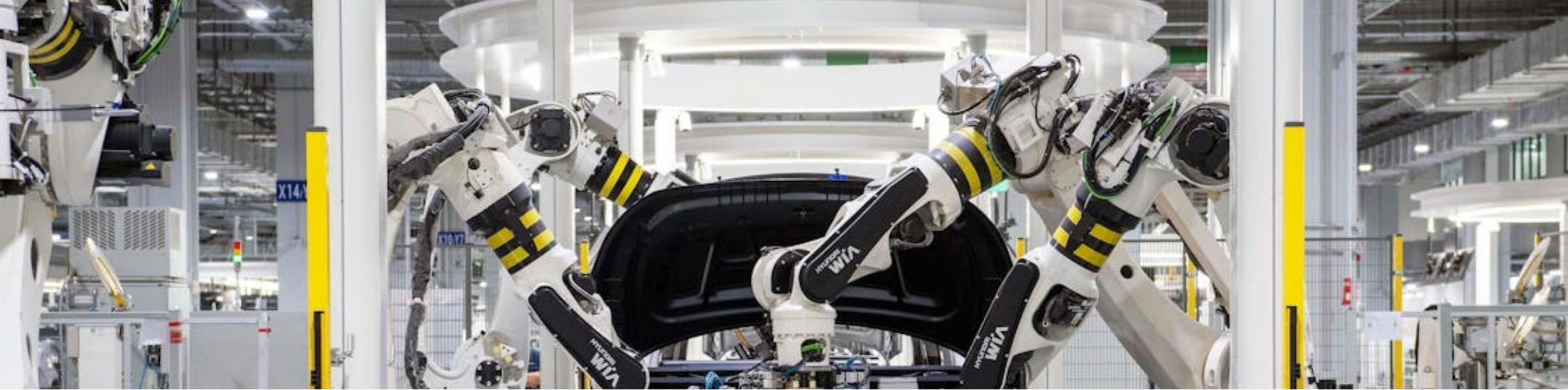
ASE-C03-01-CICD

26.1

Prof. Dr. Jens Reinhardt

University of Applied Sciences Mainz

University of Applied Sciences



CI/CD [C03-1]

Advanced Software Engineering

Prof. Dr. Jens Reinhardt

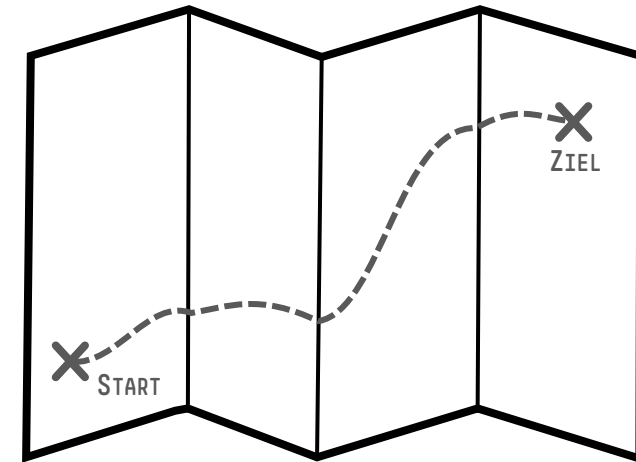


WIRTSCHAFT
HOCHSCHULE MAINZ
UNIVERSITY OF
APPLIED SCIENCES



Überblick

- Konzepte, Ziele und Nutzen...
- CI/CD-Pipeline im Überblick...
- CI/CD-Pipeline im Detail...
- Tools und Werkzeuge im Überblick...
- Best Practices und Häufig Fehler...
- Ausgesuchte Tools und Werkzeuge...



Die Anfänge der Softwareentwicklung (1960er–1990er)

- Traditionelle Softwareentwicklungsmethoden wie das Wasserfallmodell.
- Lange Release-Zyklen und manuelle Tests.
- Fokus auf Big-Bang-Releases und deren Herausforderungen.
- Erste Schritte in Richtung Versionskontrolle (z.B. *RCS*, *CVS*).

Die Einführung agiler Methoden (1990er–2000er)

- Paradigmenwechsel hin zu iterativer und inkrementeller Softwareentwicklung.
- Entstehung des Agile Manifests (2001) und Aufstieg agiler Methoden (*Scrum*, *XP* etc.).
- Einführung des Konzepts der *Continuous Integration* (Martin Fowler, 2000) und seine Grundprinzipien:
 - Regelmäßiges Zusammenführen von Code-Änderungen.
 - Automatisierte Builds und Tests.
- Erste CI-Tools wie CruiseControl.



Die Evolution zu Continuous Delivery (2000er–2010er)

- Weiterentwicklung von CI zu *Continuous Delivery* (CD).
- Fokus auf *Automatisierung* der gesamten Delivery-Pipeline.
- Entstehung von Konzepten wie automatisierte *Deployment-Pipelines* und Infrastruktur als Code.
- Verbreitung von Tools wie *Jenkins*, *TeamCity* und *Bamboo*.



Der Aufstieg von DevOps und Continuous Deployment (2010er)

- Entstehung der *DevOps*-Bewegung und die Überbrückung der Kluft zwischen Entwicklung und Betrieb.
- Einführung von *Continuous Deployment* als Erweiterung von CD.
- Kulturwandel hin zu mehr Zusammenarbeit und gemeinsamen Verantwortlichkeiten.
- Einführung von Tools wie *Docker*, *Kubernetes* und CI/CD-Integration in Cloud-Plattformen (*AWS*, *Azure*, *GCP*).

Was ist DevOps?

Unter *DevOps* versteht man diverse *Praktiken, Tools* und eine *Kulturphilosophie*, die die Prozesse zwischen Softwareentwicklungs- und IT-Teams automatisieren und integrieren. Im Vordergrund stehen dabei Teambefähigung, teamübergreifende Kommunikation und Zusammenarbeit sowie Technologieautomatisierung.

Was ist DevOps?

Die DevOps-Bewegung begann um das Jahr 2007, als die Communitys für Softwareentwicklung und IT Operations Bedenken über das traditionelle Softwareentwicklungsmodell äußerten, bei dem die Entwickler, die den Code schrieben, getrennt von Operations-Teams arbeiteten, die den Code bereitstellten und unterstützten.

Was ist DevOps?

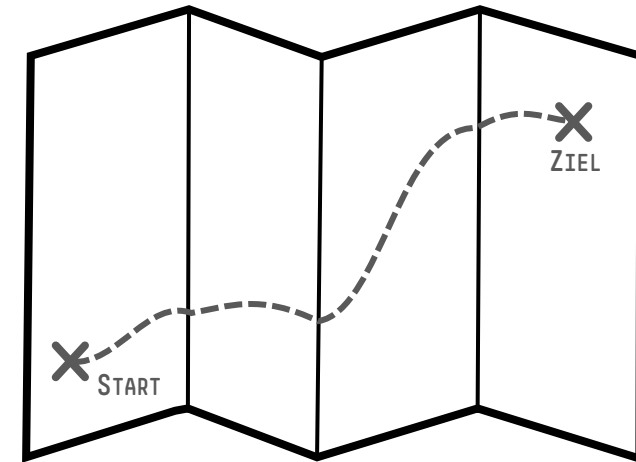
Der Begriff **DevOps**, eine Kombination aus den Wörtern **Development** und **Operations**, spiegelt den Prozess der Integration beider Fachgebiete in einen kontinuierlichen Prozess wider.

CI/CD-Praktiken und die Zukunft (2020er und darüber hinaus)

- Aktuelle Trends in CI/CD wie *Infrastructure as Code* und *observability*-getriebene Pipelines.
- Sicherheitsaspekte und die Integration von *DevSecOps*.
- Bedeutung von CI/CD für skalierbare und agile Softwareentwicklung.
- Zukunftsaussichten: *Künstliche Intelligenz* und *Machine Learning* in CI/CD, sowie autonome Pipelines.

Überblick

- ✓ Konzepte, Ziele und Nutzen...
- CI/CD-Pipeline im Überblick...
- CI/CD-Pipeline im Detail...
- Tools und Werkzeuge im Überblick...
- Best Practices und Häufig Fehler...
- Ausgesuchte Tools und Werkzeuge...



Was ist eine CI/CD-Pipeline?

CI: Continuous Integration

CD: Continuous Delivery (*und* anschliessendem Continuous Deployment)

Eine *CI/CD-Pipeline* ist eine automatisierte Abfolge von Schritten, die den Prozess der Softwareentwicklung von der Code-Integration bis hin zur Bereitstellung auf Produktionsumgebungen steuert. Sie ermöglicht es, Änderungen am Code *kontinuierlich* zu integrieren, zu testen und zu veröffentlichen.

Bestandteile einer CI/CD-Pipeline im Überblick

Die CI/CD-Pipeline besteht in der Regel aus mehreren Phasen:

1. Continuous Integration (CI)
 2. Continuous Deployment and Delivery (CD)
-

Durch die Automatisierung dieser Phasen wird die Entwicklungszeit verkürzt, die Qualität gesteigert und die Wahrscheinlichkeit von Fehlern in der Produktion reduziert.

Was ist Continuous Integration?

Bei *Continuous Integration* werden Codeänderungen, die von verschiedenen Entwicklern eingebracht werden, regelmäßig zusammengeführt.

Automatisierte Build-Prozesse und *Tests* (wie Unit-, Integrations- oder Akzeptanztests) und *Code-Analysen* stellen sicher, dass der neue Code funktioniert und keine vorhandenen Funktionen beeinträchtigt.

„Delivery liefert, Deployment bringt's live.“

deliver (sb./sth.)  *Verb* (delivered, delivered)

(etw.^{Akk}) **liefern** *v*  

We can deliver any package within five working days.
The courier delivered the package.

Wir liefern jedes Paket innerhalb von fünf Arbeitstagen.
Der Kurier hat das Paket geliefert.

etw.^{Akk} **übermitteln** *v*  

The judge delivered his verdict.

Der Richter übermittelte sein Urteil.

(etw.^{Akk}) **abliefern** *v*  

He has until ten to deliver his work.

Er hat Zeit bis zehn, um seine Arbeit abzuliefern.

„Delivery liefert, Deployment bringt's live.“

deploy (sb./sth.)  Verb (deployed, deployed)

(etw.^{Akk}) einsetzen  

The troops were deployed on a peacekeeping mission.

Die Truppen wurden bei einer Friedensmission eingesetzt.

stationieren   (Militär)

These troops are deployed there to support the local population.

Diese Truppen sind dort stationiert, um die örtliche Bevölkerung zu unterstützen.

anwenden  

It is important for a company to deploy resources efficiently.

Es ist wichtig für eine Firma, Ressourcen effizient anzuwenden.

seltener:

etw.^{Akk} bereitstellen   · etw.^{Akk} entfalten   · aufstellen   · verteidigen 

formieren  (Militär) · aufmarschieren lassen  (Militär) · aufbieten 

Delivery vs. Deployment

Continuous Delivery

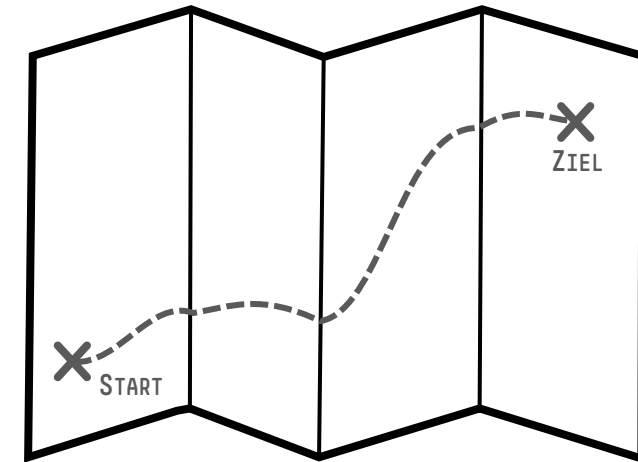
Bedeutet, dass die Software nach erfolgreichem Durchlauf aller Tests automatisiert in eine für die Bereitstellung geeignete Umgebung überführt wird. Die Freigabe in die Produktionsumgebung erfolgt manuell.

Continuous Deployment

Geht einen Schritt weiter und sorgt dafür, dass die getestete Software nach der Freigabe automatisch in die Produktionsumgebung überführt wird.

Überblick

- ✓ Konzepte, Ziele und Nutzen...
- ✓ CI/CD-Pipeline im Überblick...
- CI/CD-Pipeline im Detail...
- Tools und Werkzeuge im Überblick...
- Best Practices und Häufig Fehler...
- Ausgesuchte Tools und Werkzeuge...



Umgebungen in der professionellen Entwicklung

- *Development Environment (DEV)*

In der Entwicklungsumgebung (DEV) werden Softwareanforderungen umgesetzt und getestet. Um Fehlentwicklungen zu vermeiden, sollte sie der Produktivumgebung ähneln. Typischerweise werden alle abhängigen Dienste lokal bereitgestellt.

- *Build Environment (BUILD)*

In der Build-Umgebung (BUILD) wird der Quellcode zentral kompiliert, jedoch nicht ausgeführt. Sie muss lediglich die zum Kompilieren nötigen Abhängigkeiten bereitstellen und muss der Produktivumgebung nicht entsprechen.

Umgebungen in der professionellen Entwicklung

- *Testing Environment (TEST)*

In der Testumgebung (TEST) werden automatisierte Softwaretests durchgeführt.

Unittests laufen meist nach jedem Check-In (Versionsmanagementsystem), während lang laufende Integrationstests je nach Dauer täglich (Nightly) oder wöchentlich ausgeführt werden.

- *Quality Assurance Environment (QS/QA)*

In der Qualitätssicherungsumgebung (QA) wird die Anwendung für die Qualitätssicherung bereitgestellt. Tester suchen explorativ nach Fehlern und implementieren zusätzliche Tests. Die QA-Umgebung sollte der Produktivumgebung möglichst ähnlich sein.

Umgebungen in der professionellen Entwicklung

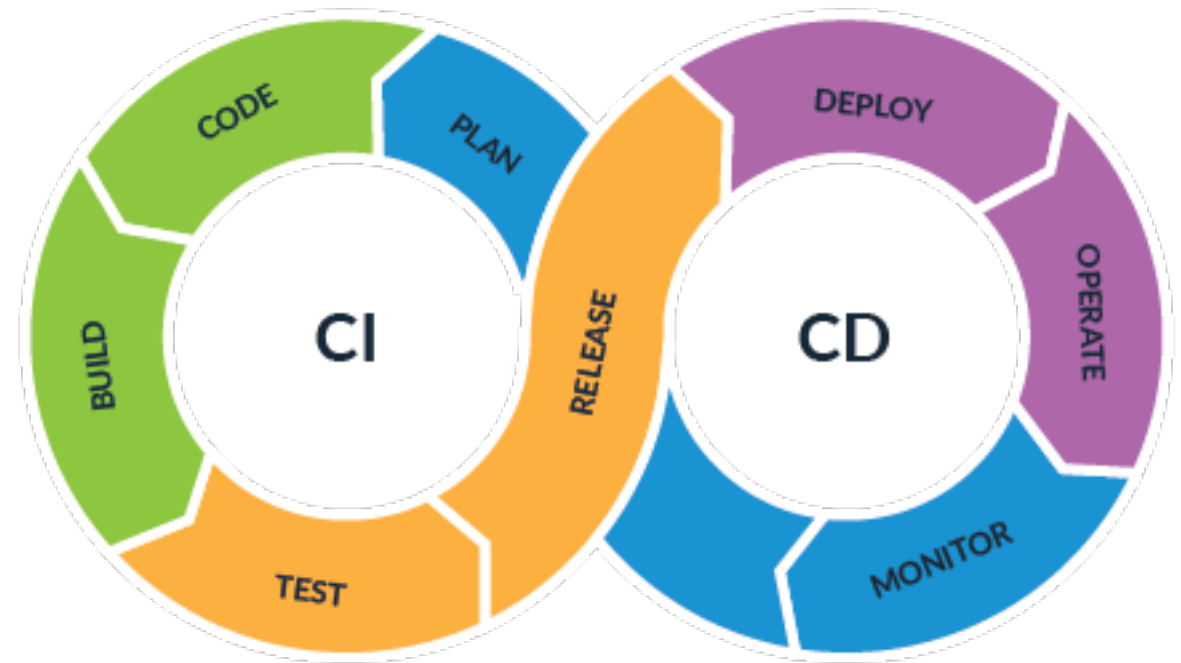
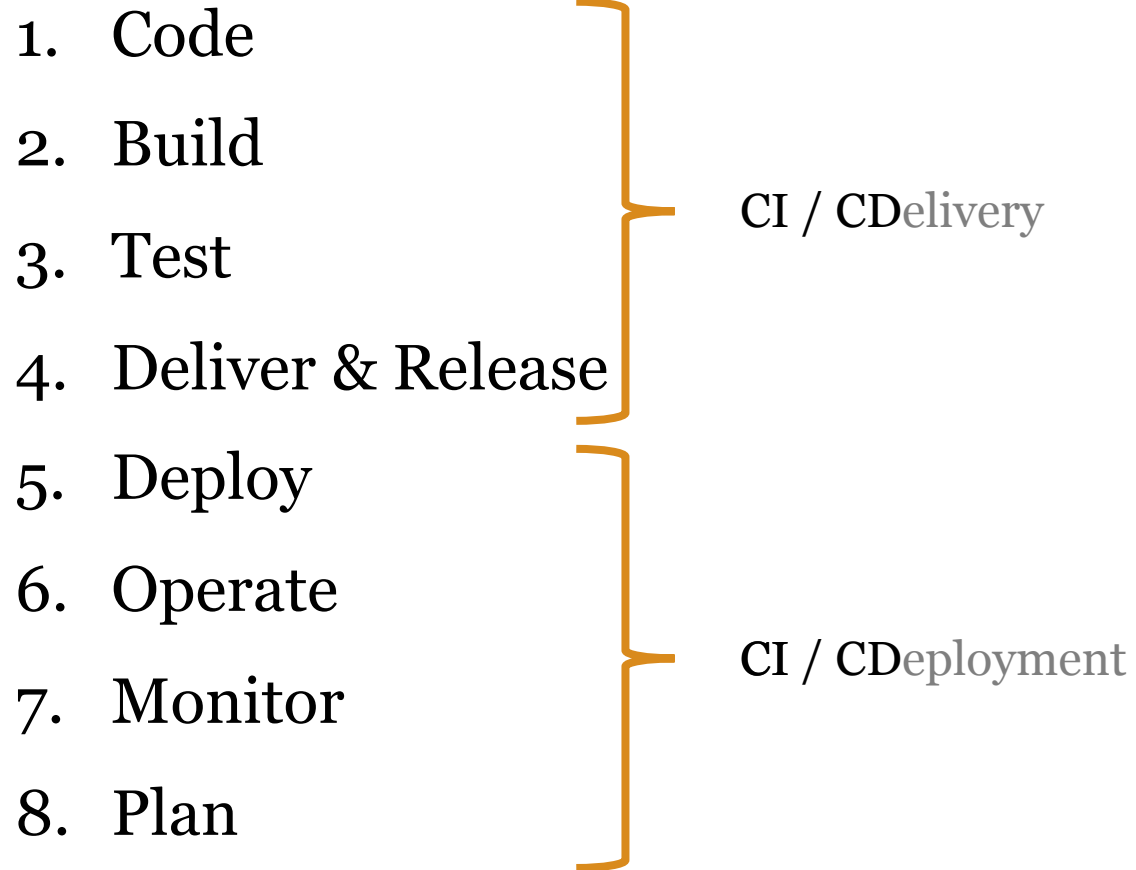
- *Staging-Environment*

Die Staging-Umgebung (Staging Environment) dient der Bereitstellung des Programms für die Abnahme durch den Product Owner und den Kunden. Sie sollte der Produktivumgebung möglichst ähnlich sein.

- *Produktivumgebung (PROD)*

In der Produktionsumgebung (PROD) wird das Programm für den eigentlichen Kundeneinsatz verwendet und generiert Geschäftswert. Die Funktion des Programms wird kontinuierlich durch Logging, Monitoring und Auditing überwacht.

Kernschritte in einer CI/CD-Pipeline



Kernschritte in einer CI/CD-Pipeline

1. Code (Entwicklung & Versionierung)

2. Build (Automatisches Compilieren)

3. Test (Automatisches Testen)

4. Deliver & Release (Bereitstellen)

5. Deploy (In die Produktion übertragen)

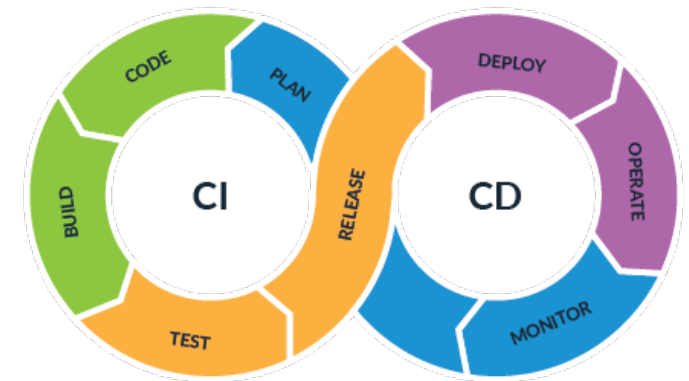
6. Operate (Betreiben)

7. Monitor (Kontinuierliche Überwachen)

8. Plan (Agile Weiterentwicklung)

CI / CDelivery

CI / CDeployment



Kernschritte in einer CI/CD-Pipeline

1. Code

2. Build

3. Test

4. Deliver & Release

5. Deploy

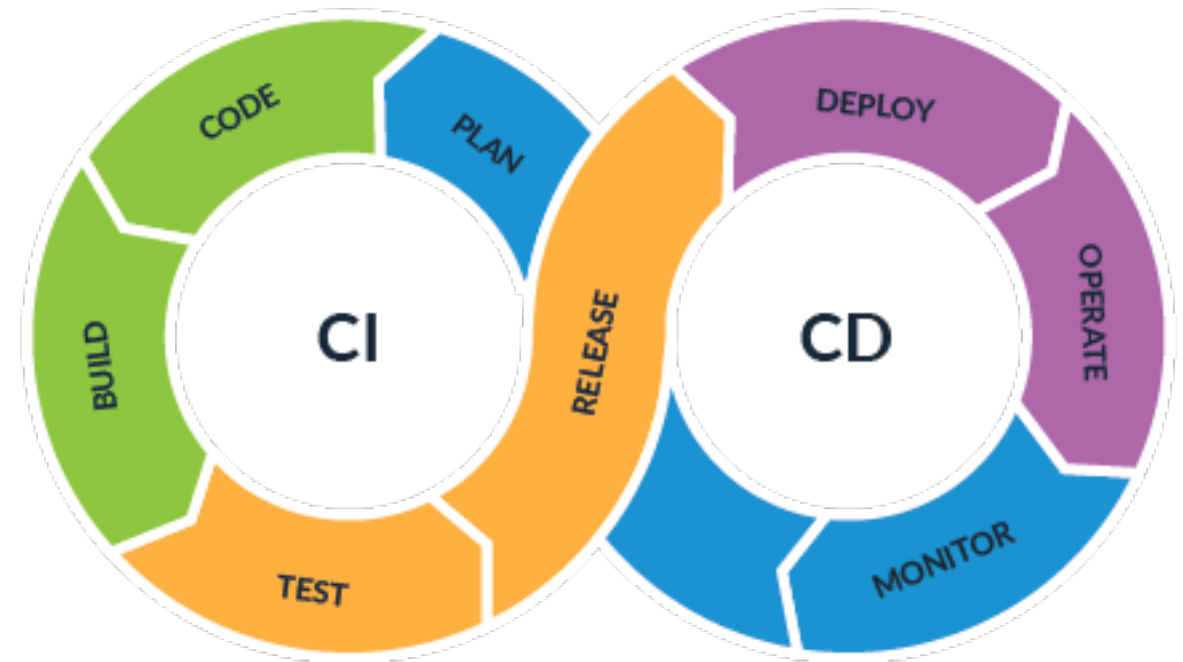
6. Operate

7. Monitor

8. Plan

CI / CDelivery

CI / CDeployment

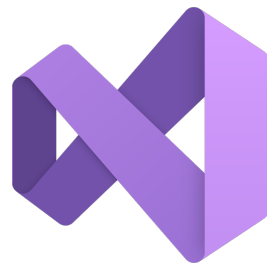


Kernschritte in einer CI/CD-Pipeline 1/8

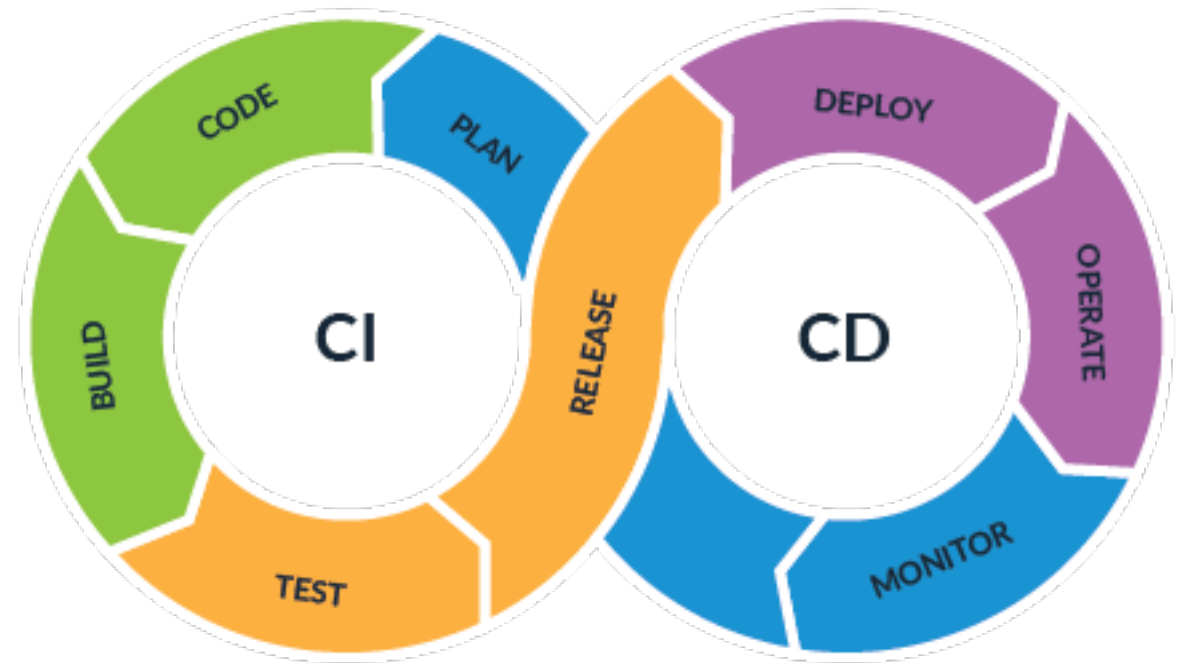
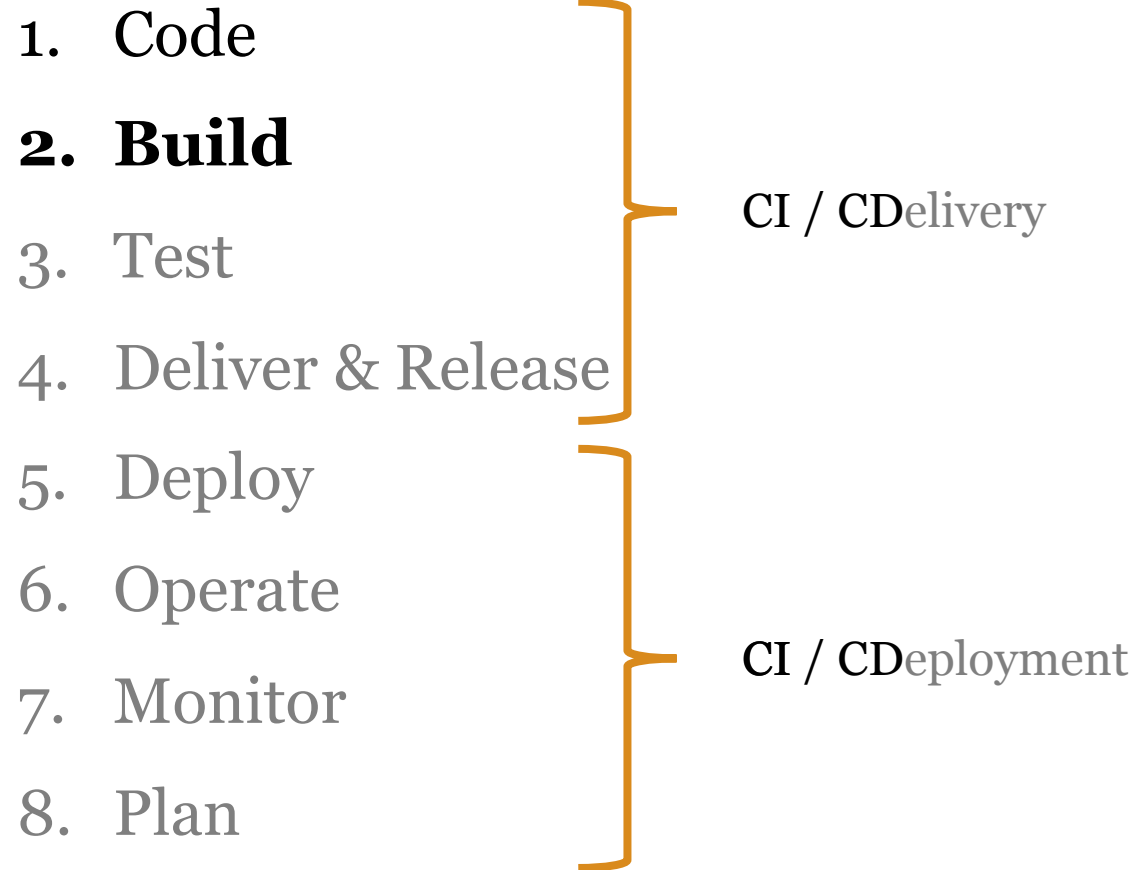
Code

Alle Änderungen am Code werden in ein Versionskontrollsystem (wie *git*) eingchecked und versioniert.

Dies ist der Ausgangspunkt für jede CI/CD-Pipeline, da alle nachfolgenden Schritte auf dem aktuellen Stand des Codes basieren.



Kernschritte in einer CI/CD-Pipeline



Kernschritte in einer CI/CD-Pipeline 2/8

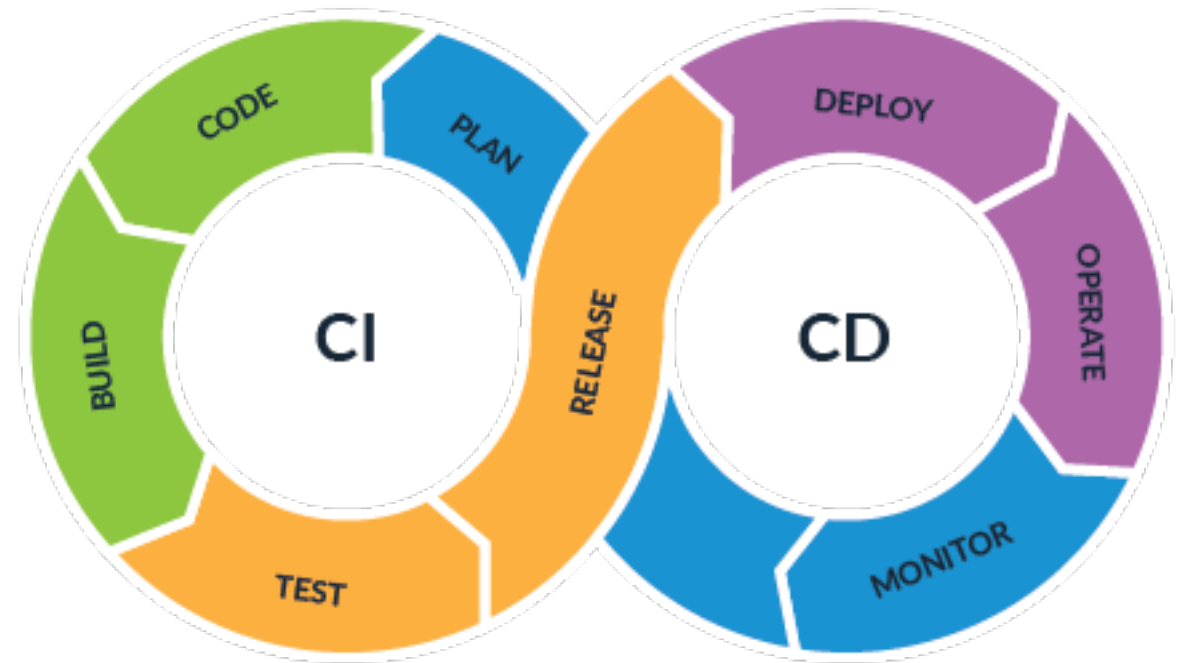
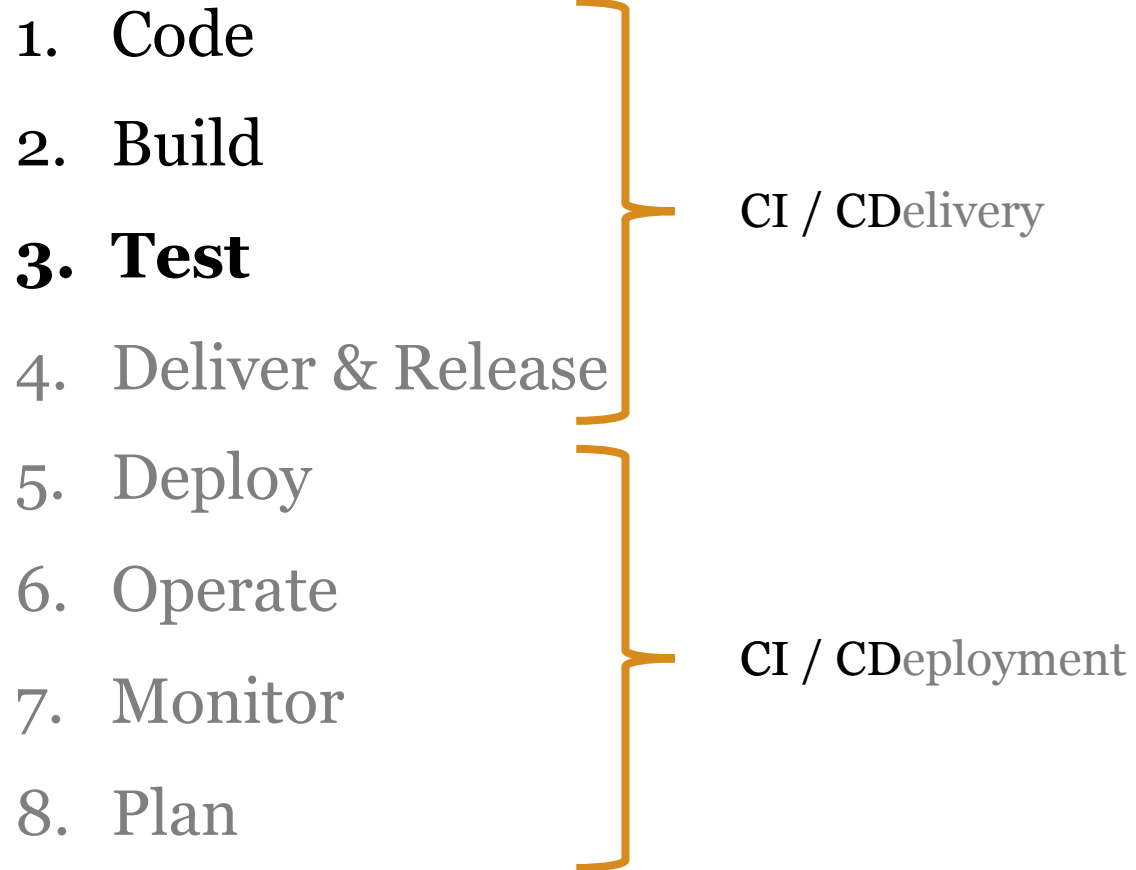
Build

In diesem Schritt wird der Quellcode in eine ausführbare Form (z. B. Binärdateien) umgewandelt.

Hier wird der Code *kompiliert*, Abhängigkeiten werden eingebunden, und Artefakte werden erstellt.

Ein erfolgreicher Build signalisiert, dass der Code *syntaktisch* korrekt ist.

Kernschritte in einer CI/CD-Pipeline



Kernschritte in einer CI/CD-Pipeline 3/8

Test & Quality I

Nach dem Build wird eine Reihe automatisierter Tests durchgeführt, um die Qualität des Codes sicherzustellen.

Dies umfasst *Unit-Tests*, *Integrations-Tests* und gegebenenfalls *Akzeptanztests*. Diese Tests sind entscheidend, um sicherzustellen, dass der neue Code bestehende Funktionen nicht bricht.

Kernschritte in einer CI/CD-Pipeline 3/8

Test & Quality II

In einigen Pipelines wird eine *statische Code-Analyse* oder ein *Code-Linter* durchgeführt, um sicherzustellen, dass der Code den Qualitätsstandards und Konventionen entspricht.

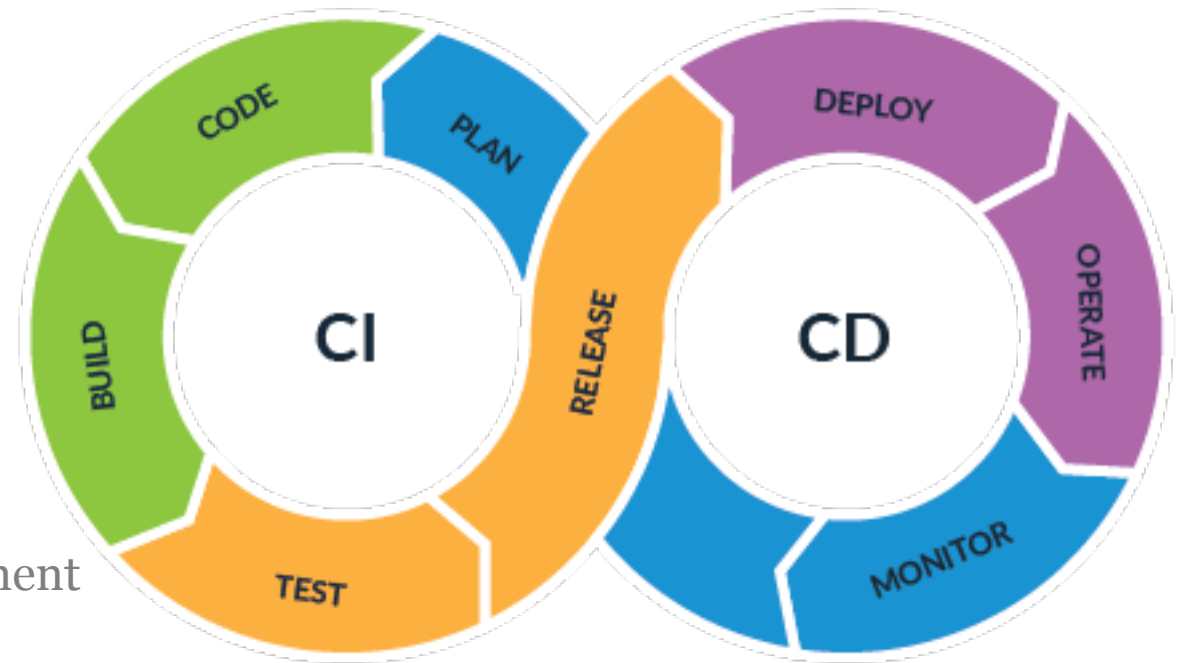
Dieser Schritt identifiziert *potenzielle* Schwachstellen, Sicherheitslücken und Code-Duplikate.

Kernschritte in einer CI/CD-Pipeline

1. Code
2. Build
3. Test
- 4. Deliver & Release**
5. Deploy
6. Operate
7. Monitor
8. Plan

CI / CDelivery

CI / CDeployment



Kernschritte in einer CI/CD-Pipeline 4/8

Delivery

Nach erfolgreicher Testdurchführung wird das Ergebnis des Builds in eine *Staging*- oder gesonderten *Testumgebung* bereitgestellt.

Diese Umgebung simuliert die Produktionsumgebung, um reale Tests durchzuführen, bevor die Software tatsächlich veröffentlicht wird.

Kernschritte in einer CI/CD-Pipeline 4/8

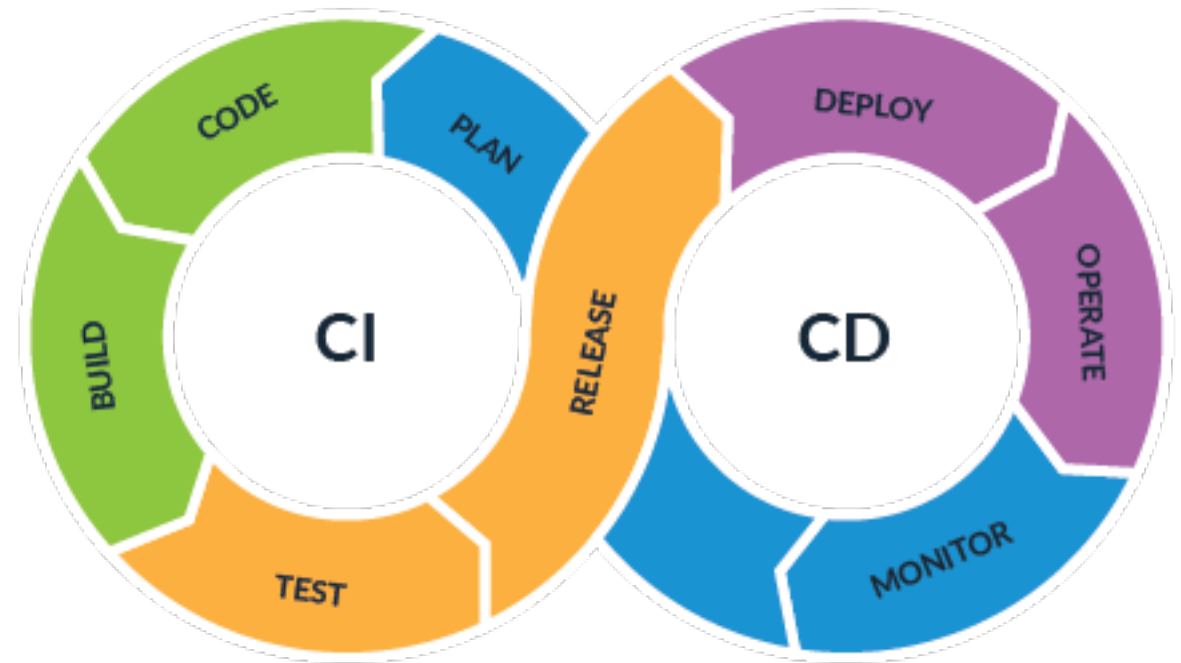
Release

In vielen Pipelines gibt es nach der Staging-Bereitstellung eine manuelle Überprüfung durch QA-Teams oder Stakeholder, bevor die endgültige Freigabe erteilt wird.

Dies ist besonders im *Continuous Deployment-Ansatz* eine entscheidende Stelle.

Kernschritte in einer CI/CD-Pipeline

1. Code
 2. Build
 3. Test
 4. Deliver & Release
 - 5. Deploy**
 6. Operate
 7. Monitor
 8. Plan
- CI / CDelivery
- CI / CDeployment



Kernschritte in einer CI/CD-Pipeline 5/8

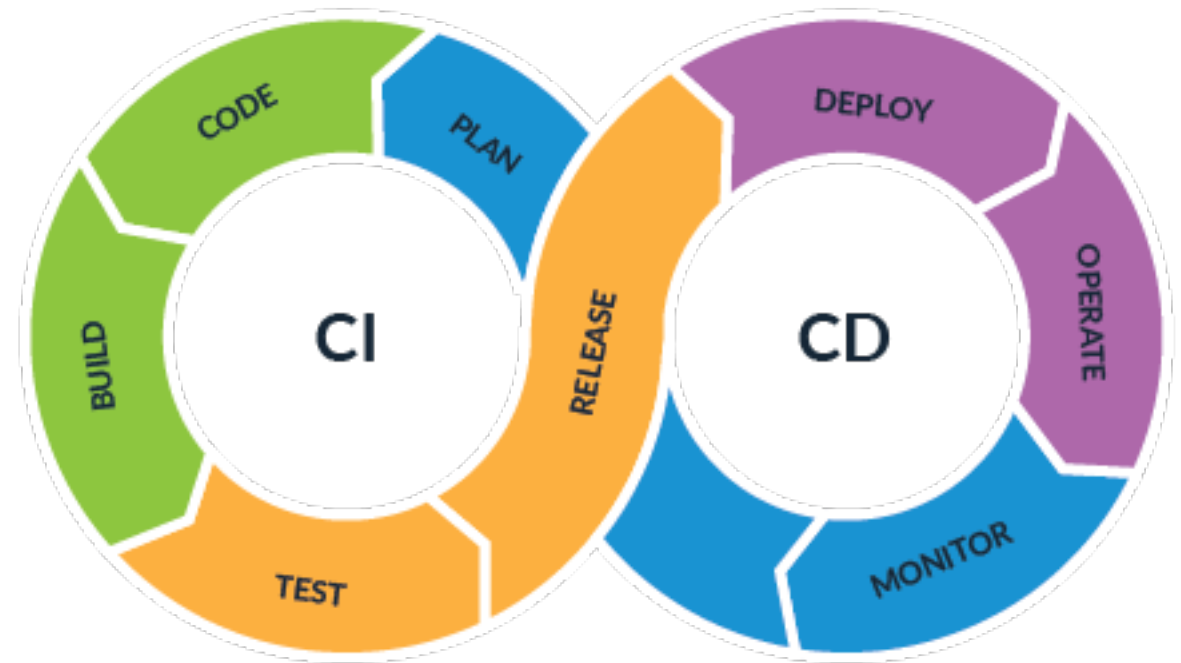
Deployment - Bereitstellung in Produktionsumgebung

Bei Continuous Deployment wird die Anwendung nach erfolgreicher Durchführung aller Tests und Genehmigungen *automatisch* in die Produktionsumgebung überführt.

Bei Continuous *Delivery* geschieht dies nach manueller Freigabe.

Kernschritte in einer CI/CD-Pipeline

1. Code
 2. Build
 3. Test
 4. Deliver & Release
 5. Deploy
 - 6. Operate**
 7. Monitor
 8. Plan
- CI / CDelivery
- CI / CDeployment



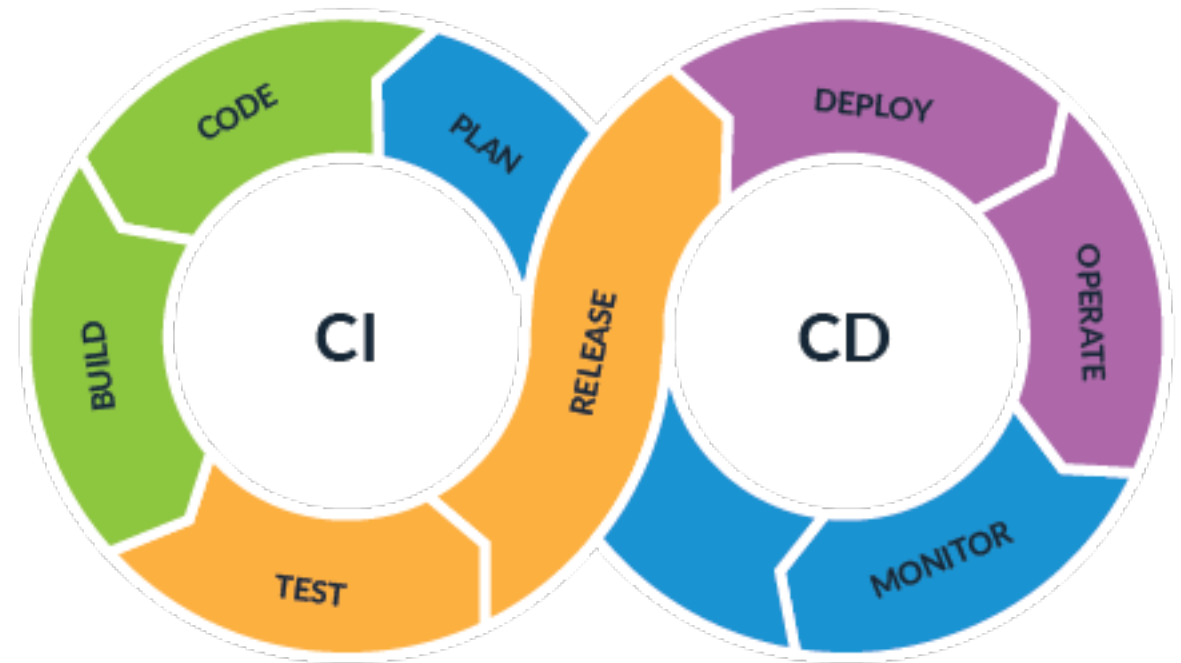
Kernschritte in einer CI/CD-Pipeline 6/8

Operate – Einsetzen einer Lösung in der Produktionsumgebung

Die Software wird in der Produktionsumgebung betrieben

Kernschritte in einer CI/CD-Pipeline

1. Code
 2. Build
 3. Test
 4. Deliver & Release
 5. Deploy
 6. Operate
 7. **Monitor**
 8. Plan
- CI / CDelivery
- CI / CDeployment

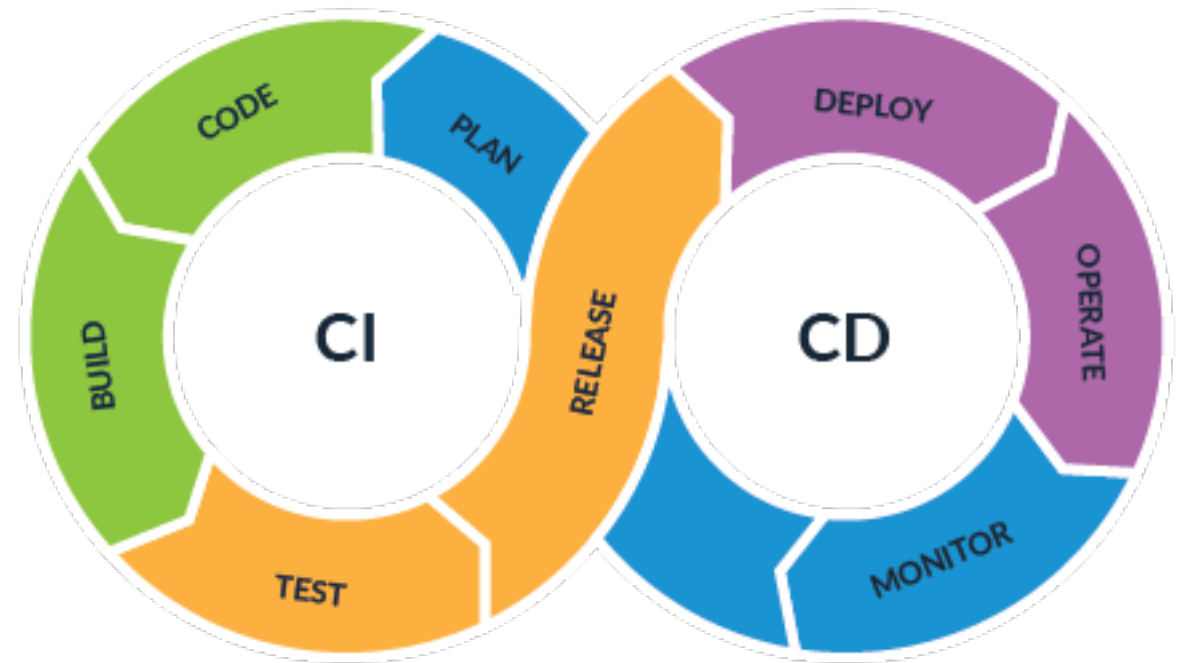
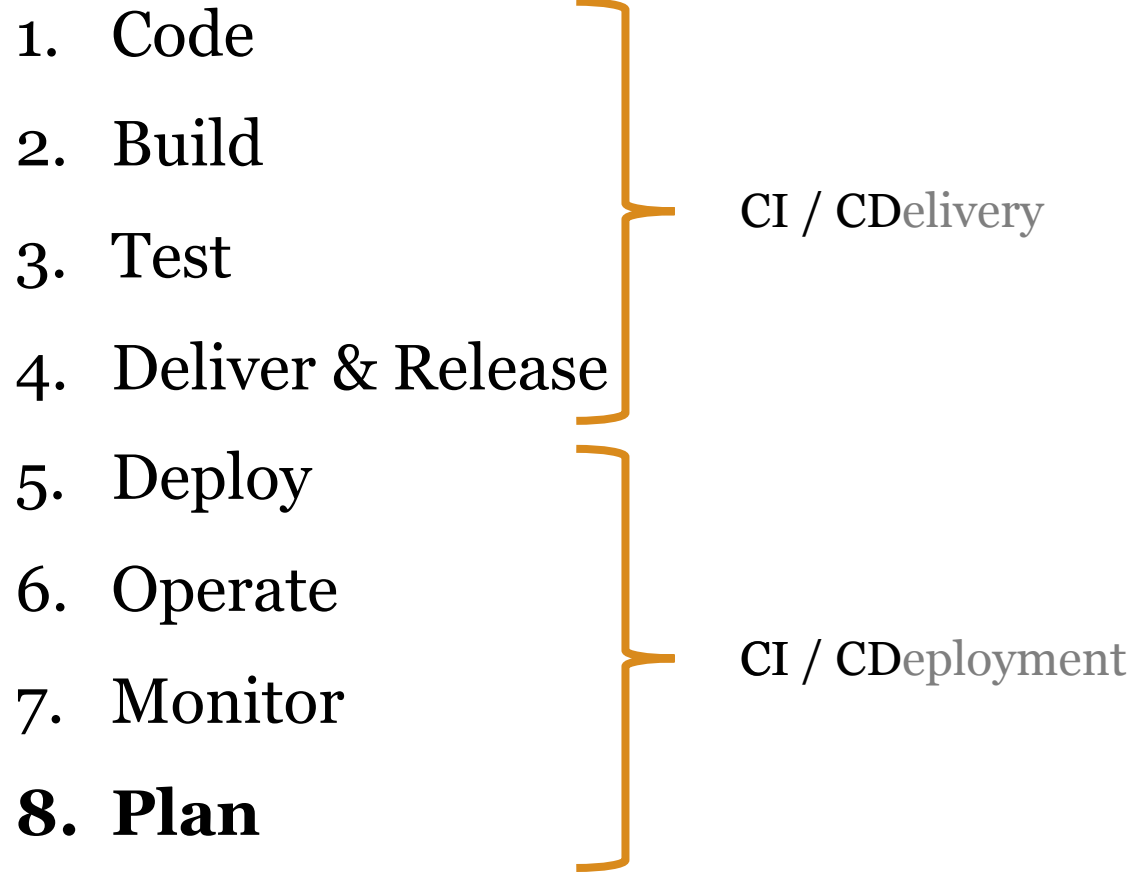


Kernschritte in einer CI/CD-Pipeline 6/8

Monitoring - Überwachung und Feedback

Nach der Bereitstellung wird die Anwendung überwacht, um sicherzustellen, dass sie stabil läuft und keine Probleme auftreten. Feedback aus der Überwachung und aus Benutzerberichten wird gesammelt, um weitere Verbesserungen zu planen.

Kernschritte in einer CI/CD-Pipeline

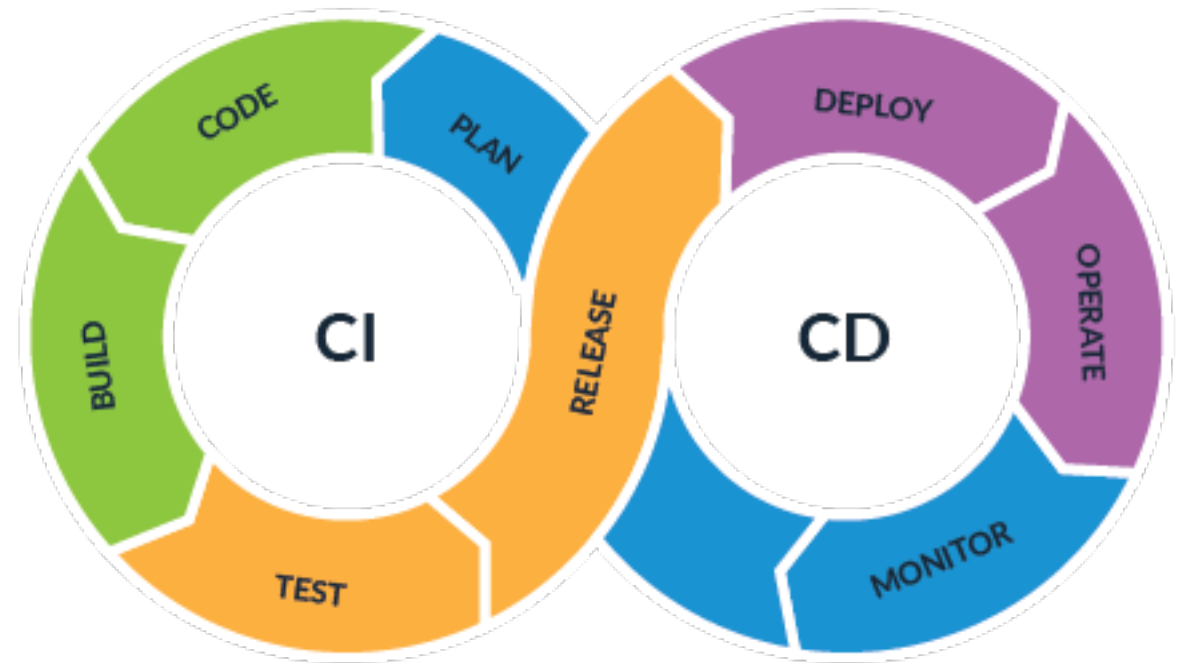
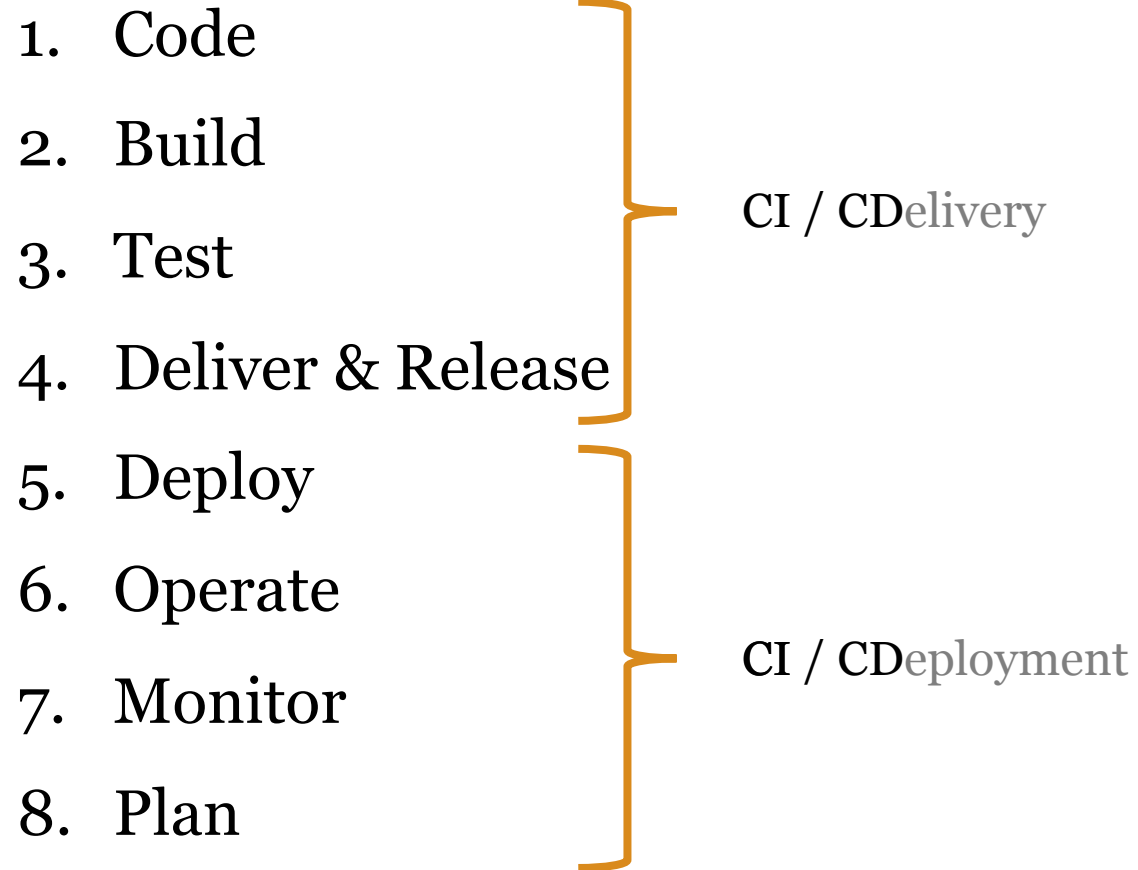


Kernschritte in einer CI/CD-Pipeline 7/8

Plan

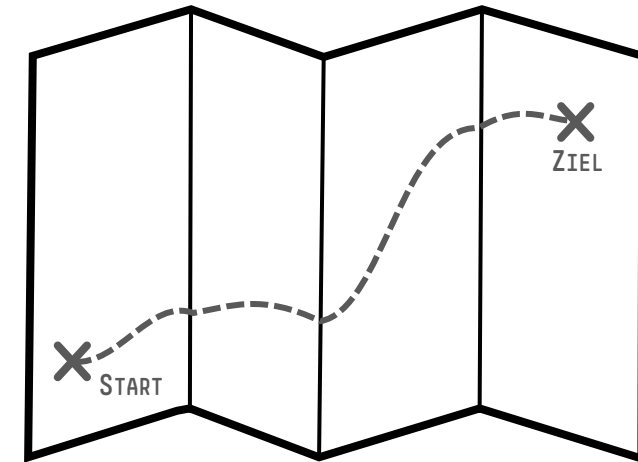
Basierend auf den Kundenwünschen und den Ergebnissen des Monitorings werden die nächsten Schritte geplant.

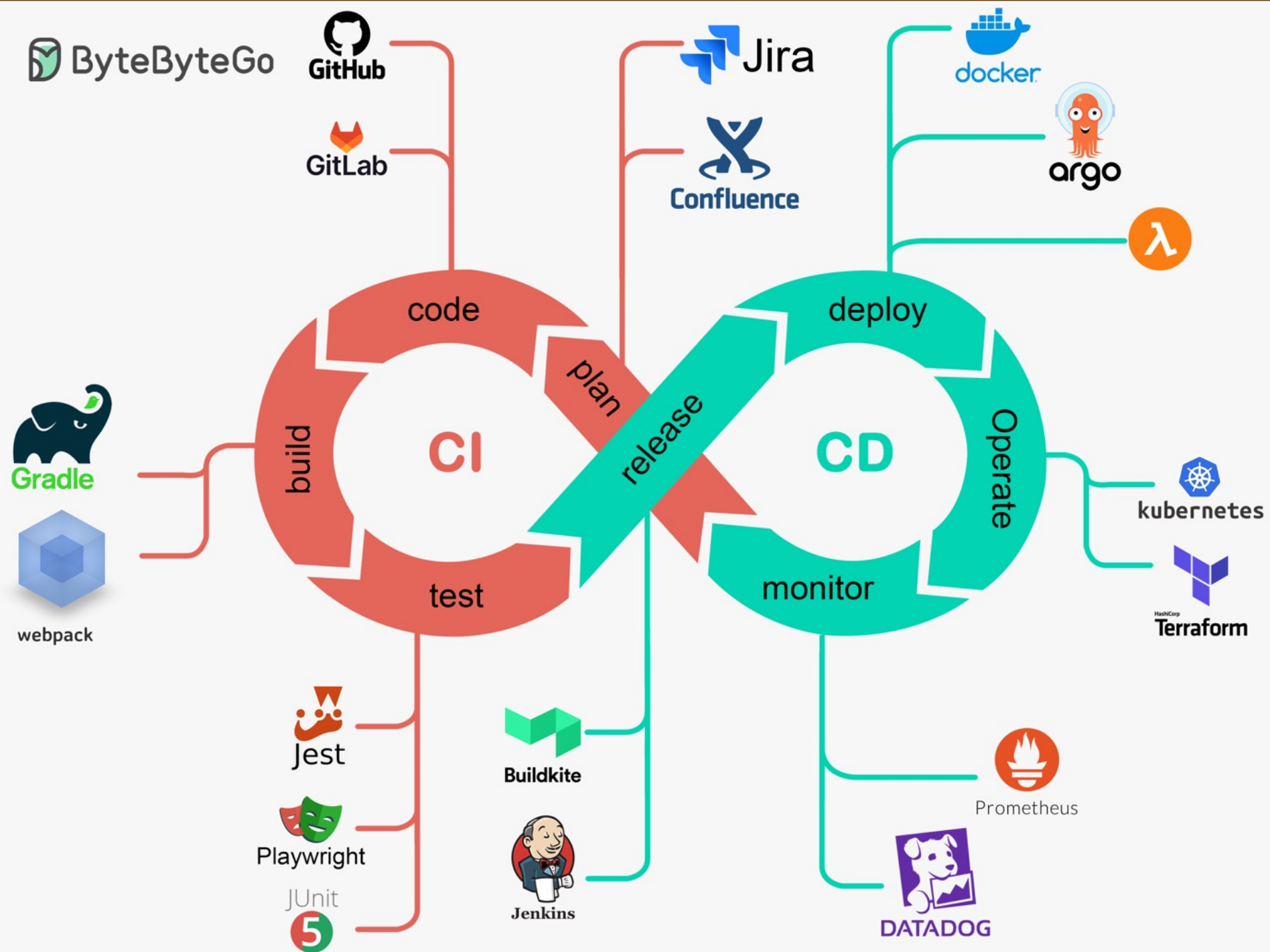
Kernschritte in einer CI/CD-Pipeline



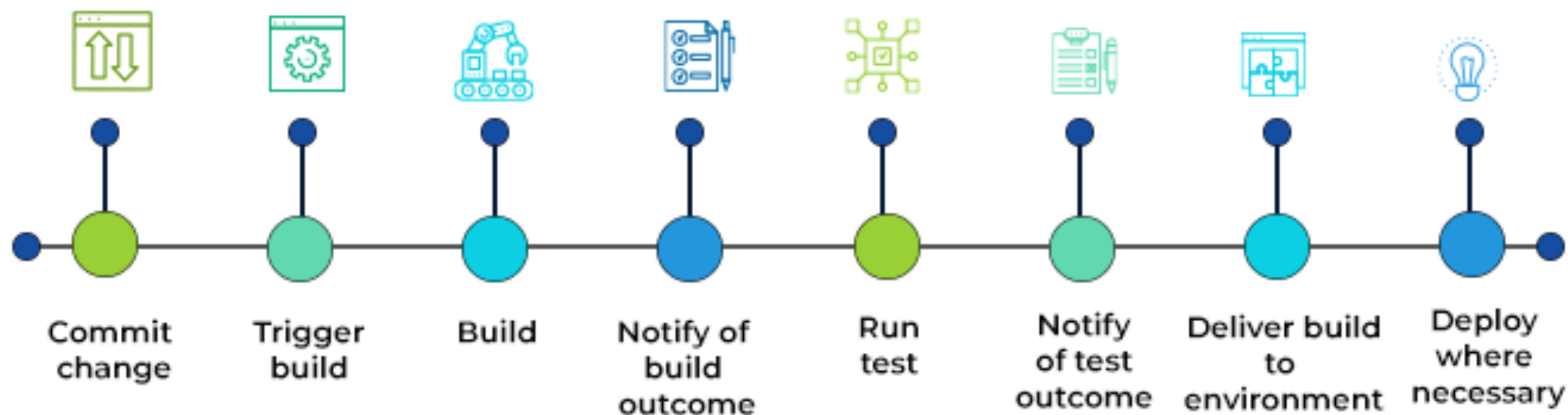
Überblick

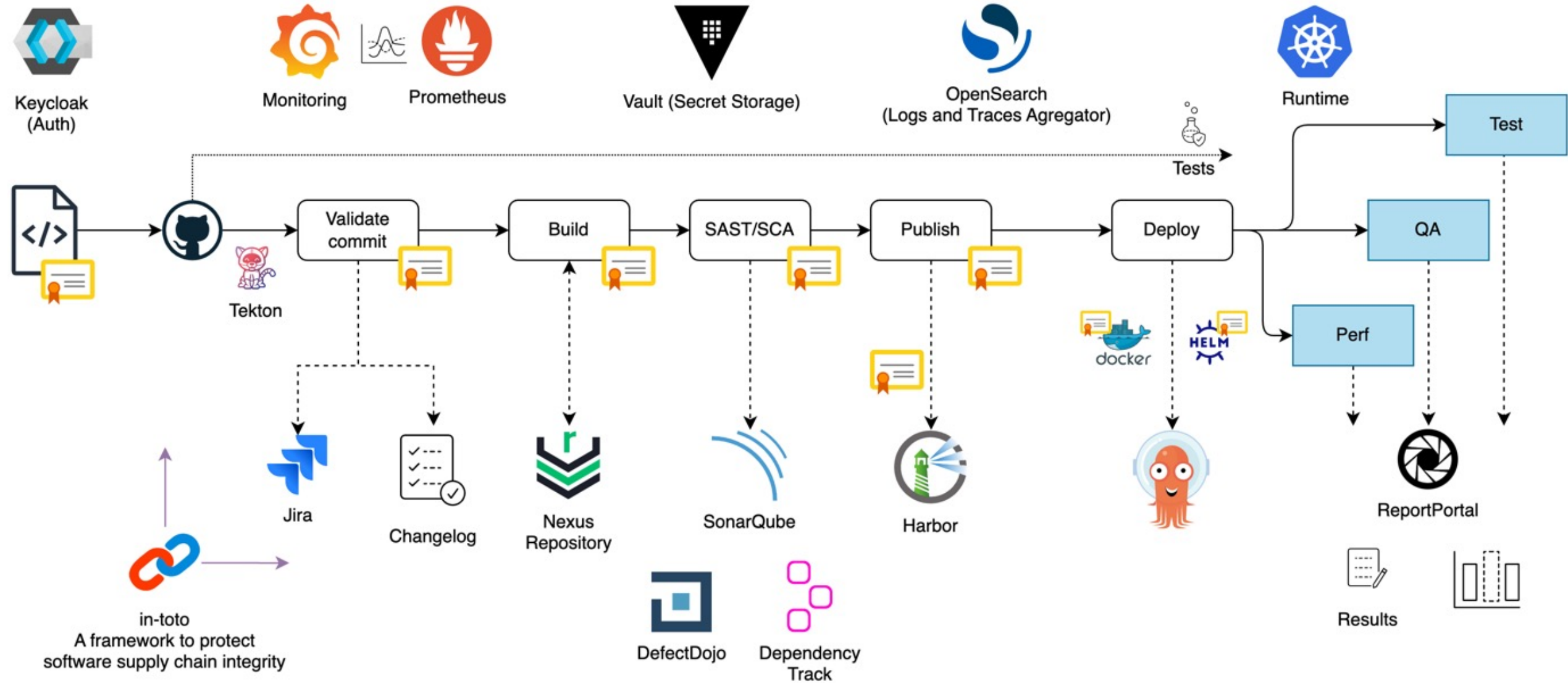
- ✓ Konzepte, Ziele und Nutzen...
- ✓ CI/CD-Pipeline im Überblick...
- ✓ CI/CD-Pipeline im Detail...
- Tools und Werkzeuge im Überblick...
- Best Practices und Häufig Fehler...
- Ausgesuchte Tools und Werkzeuge...





CI/CD PIPELINE





Collaborate

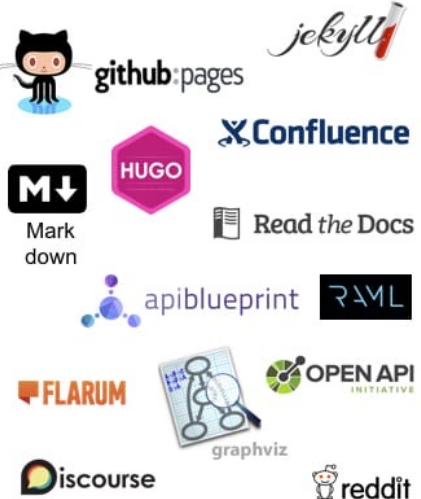
Application Lifecycle Mgmt.



Communication & ChatOps



Knowledge Sharing



Build

SCM/VCS



CI



Build



Database Management



Test

Testing



Deploy

Deployment



Config Mgmt. / Provisioning



Artefact Management



Run

Cloud / IaaS / PaaS



Orchestration & Scheduling

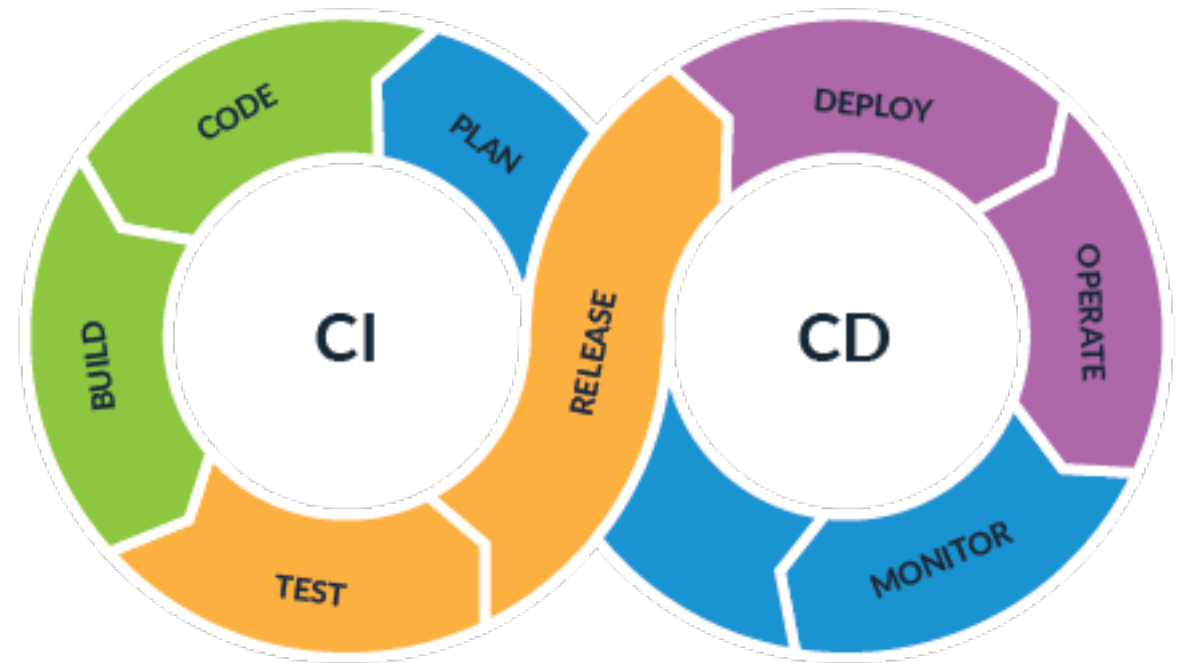


BI / Monitoring / Logging



Kernschritte in einer CI/CD-Pipeline

1. Code
 2. Build
 3. Test
 4. Deliver & Release
 5. Deploy
 6. Operate
 7. Monitor
 8. Plan
- CI / CDelivery
- CI / CDeployment



Tools und Werkzeuge im Überblick

Kernschritte in einer CI/CD-Pipeline

Code

Versionsmanagement:

Git, GitHub, GitLab, Bitbucket, Subversion (SVN)

Kernschritte in einer CI/CD-Pipeline

Build

Build-Automatisierung:

Jenkins, GitLab CI/CD, GitHub Actions, Travis CI, CircleCI, Azure Pipelines

Artifact Management:

Nexus, JFrog Artifactory, GitHub Packages, Azure Artifacts

Tools und Werkzeuge im Überblick

Kernschritte in einer CI/CD-Pipeline

Test & Quality

Automatisierte Test:

Selenium, JUnit, TestNG, Cypress, Jest, Mocha,

Tools und Werkzeuge im Überblick

Kernschritte in einer CI/CD-Pipeline

Test & Quality

Code-Qualität:
SonarQube

Kernschritte in einer CI/CD-Pipeline

Delivery & Deployment

Orchestrierung:

Kubernetes, Docker Swarm, Nomad, OpenShift

Containerisierung:

Docker, Podman, CRI-O

Infrastructure as Code (IaC):

Terraform, Ansible, Chef, Puppet, AWS CloudFormation

Deployment-Automatisierung:

ArgoCD, Spinnaker, Octopus Deploy

Kernschritte in einer CI/CD-Pipeline

Monitoring

Monitoring und Logging:

Prometheus, Grafana, ELK Stack (Elasticsearch, Logstash, Kibana), Datadog, Splunk

Security und DevSecOps:

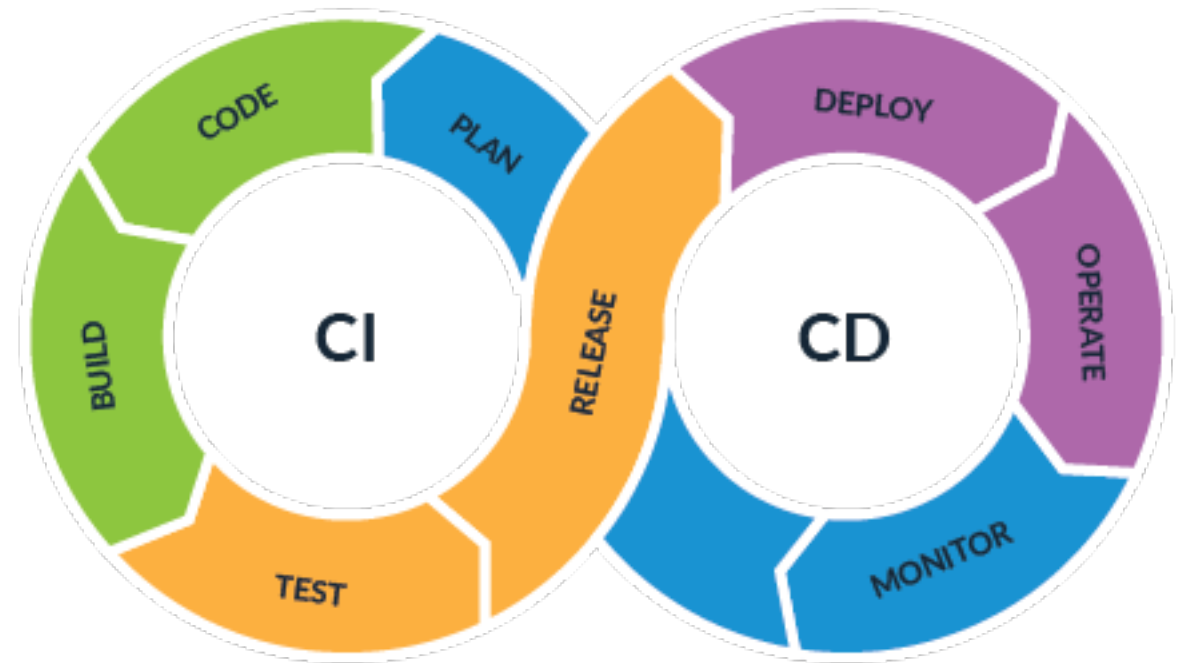
Snyk, Aqua Security, HashiCorp Vault, SonarQube (für statische Codeanalyse), Clair

Alerting und Incident-Management:

PagerDuty, Opsgenie, VictorOps, Prometheus Alertmanager

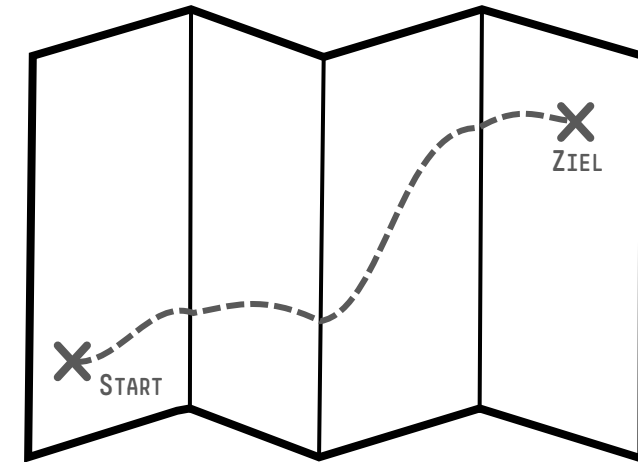
Kernschritte in einer CI/CD-Pipeline

1. Code
 2. Build
 3. Test
 4. Deliver & Release
 5. Deploy
 6. Operate
 7. Monitor
 8. Plan
- CI / CDelivery
- CI / CDeployment



Überblick

- ✓ Konzepte, Ziele und Nutzen...
- ✓ CI/CD-Pipeline im Überblick...
- ✓ CI/CD-Pipeline im Detail...
- ✓ Tools und Werkzeuge im Überblick...
- Best Practices und Häufig Fehler...
- Ausgesuchte Tools und Werkzeuge...



Häufige Fehler und Missverständnisse

1. Fehlende Tests oder unzureichende Testabdeckung
2. Manuelle Eingriffe in der Pipeline
3. Fehlende Versionskontrolle von Pipelines und Konfigurationsdateien
4. Verwendung der gleichen Umgebung für Tests und Produktion
5. Schlechte Fehlermeldungen und unzureichendes Logging

Häufige Fehler und Missverständnisse

Fehlende Tests oder unzureichende Testabdeckung

Ein häufiges Problem ist, dass nicht genügend automatisierte Tests vorhanden sind oder die Testabdeckung unzureichend ist. Ohne ausreichende Tests ist es schwer, die Stabilität der Anwendung zu gewährleisten.

Häufige Fehler und Missverständnisse

Manuelle Eingriffe in der Pipeline

Viele Teams greifen manuell in automatisierte Pipelines ein, um Fehler zu korrigieren oder Codeänderungen zu implementieren. Dies unterbricht den automatisierten Workflow und erhöht die Fehleranfälligkeit.

Häufige Fehler und Missverständnisse

Fehlende Versionskontrolle von Pipelines und Konfigurationsdateien

Es ist ein Missverständnis, dass nur der Anwendungscode versioniert werden sollte. Pipelineskripte, Konfigurationsdateien und Infrastruktur-Code müssen ebenfalls versioniert werden, um Änderungen nachverfolgen zu können.

Häufige Fehler und Missverständnisse

Verwendung der gleichen Umgebung für Tests und Produktion

Einige Teams setzen Test- und Produktionsumgebungen gleich oder testen direkt in der Produktionsumgebung. Dies kann zu unvorhergesehenen Problemen führen, wenn die beiden Umgebungen nicht identisch sind.

Häufige Fehler und Missverständnisse

Schlechte Fehlermeldungen und unzureichendes Logging

Fehlende oder unklare Fehlermeldungen in der Pipeline machen es schwierig, die Ursache eines Fehlers schnell zu identifizieren. Dies führt oft zu längeren Behebungszeiten und geringerer Effizienz.

Häufige Fehler und Missverständnisse

1. Fehlende Tests oder unzureichende Testabdeckung
2. Manuelle Eingriffe in der Pipeline
3. Fehlende Versionskontrolle von Pipelines und Konfigurationsdateien
4. Verwendung der gleichen Umgebung für Tests und Produktion
5. Schlechte Fehlermeldungen und unzureichendes Logging

Best Practices

1. Umfassende und automatisierte Tests
2. End-to-End-Automatisierung
3. Versionierung der Pipeline und Konfigurationsdateien
4. Verwendung von isolierten Staging-Umgebungen
5. Gutes Logging und Monitoring

Best Practices

Umfassende und automatisierte Tests

Implementiere eine Vielzahl automatisierter Tests (Unit-, Integrations-, und Akzeptanztests), um die Softwarequalität sicherzustellen. Achte auf eine hohe Testabdeckung und prüfe regelmäßig, ob die Tests aktuell sind.

Best Practices

End-to-End-Automatisierung

Vermeide manuelle Eingriffe, wo immer es möglich ist. Der gesamte Prozess, vom Code-Commit bis zur Bereitstellung, sollte automatisiert sein, um menschliche Fehler zu minimieren.

Best Practices

Versionierung der Pipeline und Konfigurationsdateien

Nutze Versionskontrollsysteme nicht nur für den Code, sondern auch für die CI/CD-Pipelineskripte und Konfigurationsdateien. Dies ermöglicht es, Änderungen und Fehlerquellen besser nachzuverfolgen.

Best Practices

Verwendung von isolierten Staging-Umgebungen

Teste immer in einer Staging-Umgebung, die die Produktionsumgebung so genau wie möglich simuliert. Dies hilft, unvorhergesehene Fehler in der Produktion zu vermeiden.

Best Practices

Gutes Logging und Monitoring

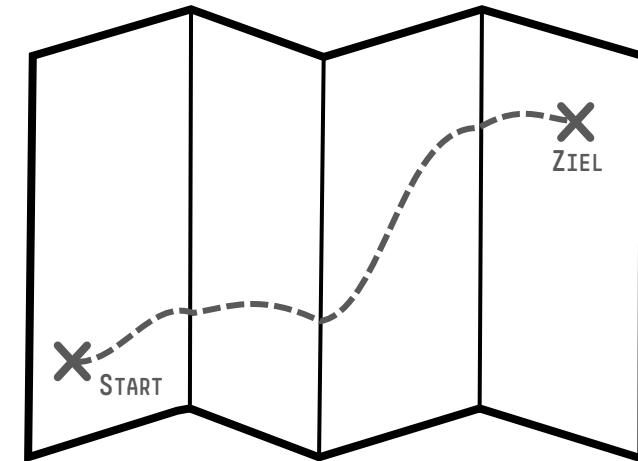
Sorge für klar verständliche und aussagekräftige Logs, die einfach zugänglich sind. Implementiere zudem eine Überwachung, um Probleme nach der Bereitstellung schnell erkennen zu können. Nutze diese Informationen für das kontinuierliche Lernen und die Verbesserung der Pipeline.

Best Practices

1. Umfassende und automatisierte Tests
2. End-to-End-Automatisierung
3. Versionierung der Pipeline und Konfigurationsdateien
4. Verwendung von isolierten Staging-Umgebungen
5. Gutes Logging und Monitoring

Überblick

- ✓ Konzepte, Ziele und Nutzen...
- ✓ CI/CD-Pipeline im Überblick...
- ✓ CI/CD-Pipeline im Detail...
- ✓ Tools und Werkzeuge im Überblick...
- ✓ Best Practices und Häufig Fehler...
- Ausgesuchte Tools und Werkzeuge...



Jenkins

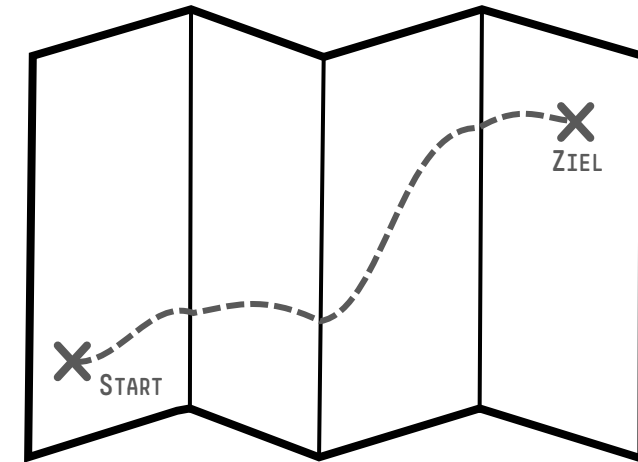


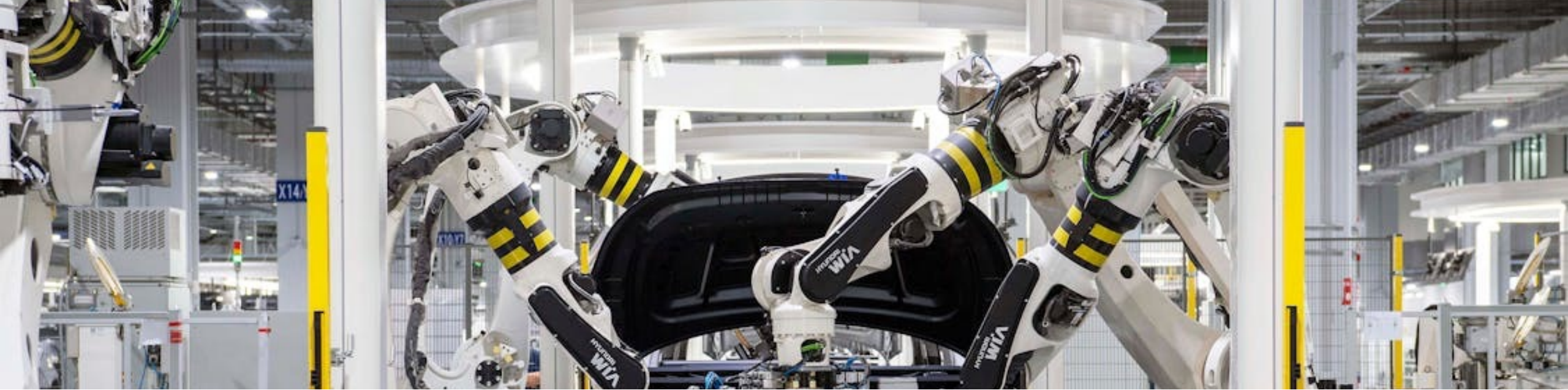
Jenkins

Beschreibung	Jenkins ist ein weit verbreitetes Open-Source-Automatisierungstool, das ursprünglich für Continuous Integration (CI) entwickelt wurde und später zu einem wichtigen Bestandteil von CI/CD-Pipelines wurde.
Einsatzzweck	Automatisierung von Builds, Tests und Deployments.
Erstellungsdatum	2011 (aus dem ursprünglichen Hudson-Projekt entstanden).
Verbreitung	2013 - 2020.
Zukunft	Trotz wachsender Konkurrenz bleibt Jenkins eine wichtige Plattform, vor allem für komplexe und anpassbare CI/CD-Pipelines.
Kritik	Komplexe Konfiguration und Benutzeroberfläche; benötigt oft viele Plugins, was zu Wartungsproblemen führen kann.
Lob	Flexibilität durch Plugins und große Community-Unterstützung.

Überblick

- ✓ Konzepte, Ziele und Nutzen...
- ✓ CI/CD-Pipeline im Überblick...
- ✓ CI/CD-Pipeline im Detail...
- ✓ Tools und Werkzeuge im Überblick...
- ✓ Best Practices und Häufig Fehler...
- ✓ Ausgesuchte Tools und Werkzeuge...





CI/CD [C03-1]

Advanced Software Engineering

Prof. Dr. Jens Reinhardt



WIRTSCHAFT
HOCHSCHULE MAINZ
UNIVERSITY OF
APPLIED SCIENCES

